Adama City Citizen Complaint System

Backend Presentation Guide

A practical guide to understanding the backend, explaining the codebase, preparing a live demonstration, and answering technical questions.

Prepared from the project source code and its documentation | October 2026

How to use this guide

This guide is designed to help you explain your backend confidently, not memorize every line. Focus on the request lifecycle, complaint workflow, role boundaries, and the configuration that makes the services run. The code paths cited below are actual project paths; use them to rehearse a short code walkthrough.

Presentation principle: explain what the system does, how the backend enforces it, and what its current limits are. Clearly distinguish implemented behavior from optional integrations and future improvements.

1. Project purpose and scope

The Adama City Citizen Complaint System gives residents a way to report and follow municipal issues. Administrators route complaints to departments or officers. Department officers work on eligible cases and report progress. The backend provides the API, identity checks, permissions, workflow rules, database persistence, history, and notifications.

The delivered product is complaint-focused; it does not provide a separate service-request module. The principal roles are citizen, administrator, and department officer.

Roles at a glance

- Citizen: register, sign in, submit complaints, view their own complaints and history, and edit a complaint only while it is pending.
- Administrator: review complaints, assign a department and optionally an officer, manage users and departments, view reports, and inspect activity logs.
- Department officer: see personally assigned complaints and eligible unassigned complaints in their department, then start work and update status.

2. System architecture

The React/Vite frontend sends HTTP requests to the Express backend. The backend authenticates and authorizes the request, validates inputs, applies workflow logic, and uses Mongoose to read or write MongoDB. External services such as the AI sidecar, email provider, and SMS provider are integrations around that core flow.

```
React frontend  ->  Express backend API  ->  MongoDB
                            |
                            +--> optional AI sidecar
                            +--> email provider (Resend / SMTP)
                            +--> Africa's Talking SMS
```

The AI service is separate and optional. It can provide writing help, triage suggestions, similar-case results, resolution drafts, and a FAQ chatbot. AI suggestions do not automatically assign, reject, resolve, or close a complaint; people make those decisions in the main application.

Where the main pieces live

- Application composition and middleware order: backend/src/app.js
- Normal Node server startup and database connection: backend/src/index.js
- MongoDB connection configuration: backend/src/config/db.js
- API route definitions: backend/src/routes/
- Request handlers and use cases: backend/src/controllers/
- MongoDB schemas: backend/src/models/
- Authentication, validation, error handling, and AI proxy: backend/src/middleware/
- Shared workflow and access rules: backend/src/utils/

- Email/SMS integrations: backend/src/services/

3. Startup and application configuration

The usual backend entry point is backend/src/index.js. It loads dotenv configuration, requires JWT_SECRET, rejects the known placeholder secret in production, connects to MongoDB, creates the Express application, and listens on PORT (default 5000). It also logs whether SMS is disabled, unconfigured, sandboxed, or live.

backend/src/app.js builds the Express app. Middleware order matters because middleware executes in the order it is registered: Helmet security headers; CORS origin checks; JSON and URL-encoded body parsing; static upload serving; AI proxy; API routes; optional production frontend serving; not-found handling; then centralized error handling.

CORS is configured using backend/src/utils/publicOrigin.js. It allows requests without a browser Origin header, the configured public origin, and local development origins when not in production. CORS is a browser cross-origin policy; it does not authenticate users and is not a replacement for API authorization.

MongoDB configuration accepts MONGODB_URI, or Atlas fields such as MONGODB_USER, MONGODB_PASSWORD, MONGODB_CLUSTER, and MONGODB_DB. If credentials are not supplied, the code has a local MongoDB fallback. The connection uses Mongoose and enables strictQuery.

4. Follow one request through the backend

```
HTTP request
  -> Express route matches method and path
  -> authenticate verifies Bearer JWT
  -> attachUser loads and checks active user
  -> authorize checks required role
  -> validators inspect request fields
  -> controller enforces business rules
  -> Mongoose reads/writes MongoDB
  -> toClient removes internal/sensitive fields
  -> JSON response
```

For example, complaint routes in backend/src/routes/complaints.routes.js apply authentication and load the user before complaint operations. Each route then declares role requirements, input validation, upload handling if needed, and its controller.

Important vocabulary

- Middleware: reusable request processing that runs before the final handler.
- Route: connects an HTTP method and URL to middleware and a controller.
- Controller: coordinates one use case, including reads, rules, writes, and response.
- Model/schema: defines document fields, types, validation, references, and indexes.
- Validation: checks whether input has the expected shape and acceptable values.
- Authorization: checks whether this authenticated user may perform an action.

5. Authentication and security controls

Authentication and authorization are separate. Authentication asks who the requester is; authorization asks what that identity is allowed to do.

backend/src/middleware/auth.js contains three key steps. authenticate reads the Authorization: Bearer token header and verifies the JWT with JWT_SECRET. attachUser loads the account from MongoDB and rejects missing or inactive users. authorize(...roles) permits only specified roles. When the database user is attached, the current database role takes precedence over the role embedded in the token, allowing role changes to take effect before token expiry.

JWT means JSON Web Token. The server signs the token so clients cannot alter its claims undetected. A JWT payload is encoded, not encrypted; do not put secrets in it. The token includes a subject, role, and email and expires according to JWT_EXPIRES_IN, default 7d.

Registration and login are handled by backend/src/controllers/authController.js. Public registration creates citizens only. Passwords are hashed using bcryptjs with 12 rounds; the plain-text password is not stored. Login compares the submitted password with the stored hash and returns a token and a safe user object.

Password reset uses a randomly generated token. The database stores a SHA-256 hash of the token plus a one-hour expiry; the raw token is sent through the reset link. The reset endpoint hashes the submitted token and checks the hash, expiry, and active-account state. Forgot-password uses a generic response so the endpoint does not reveal whether an email belongs to an account, and it has a simple in-memory per-email throttle.

backend/src/utils/toClient.js strips password hashes, reset-token data, Mongo internal fields, and normalizes reference IDs before records are sent to the frontend.

Security controls to name

- JWT verification and expiration.
- Hashed passwords rather than plain-text storage.
- Role-based route restrictions and ownership/department filters.
- Input validation and Mongoose schema validation.
- Helmet security headers and configured CORS.
- Safe client serialization that removes sensitive fields.

6. Complaint lifecycle and business rules

The statuses are pending, in_progress, resolved, rejected, and closed. Allowed transitions depend on both current status and role. The central transition table is backend/src/utils/statusTransitions.js; the controller checks it before writing a status.

- Officer: pending -> in_progress; in_progress -> resolved or closed. Officers cannot reject complaints.
- Administrator: pending -> rejected; in_progress -> resolved, closed, or rejected; resolved -> closed.
- A complaint cannot be transitioned to its current status.

Citizen submission

The POST /api/complaints route is citizen-only. It validates title, description, category, and an allowed Adama location. The controller creates a pending complaint, records the citizen as owner, generates a reference ID, and creates an initial status-history record. A photo can be attached through Multer.

Visibility and officer scope

Complaint list filters differ by role: admins can see all complaints; citizens see only records whose citizenId belongs to them; officers see records assigned to them or unassigned items in their department. backend/src/utils/officerScope.js builds the MongoDB filter and checks access to an individual complaint. Being logged in as an officer does not grant access to every complaint.

Admin routing and officer assignment

An administrator selects an active department and may select an officer who is active and belongs to that department. Routing only to a department leaves the complaint pending in that department queue. Assigning a specific officer changes it to in_progress. The backend records status history and activity and notifies the citizen and relevant officers.

Atomic officer claim

When an officer starts an unassigned department-queue task, the backend conditionally updates it only if the complaint is still pending, belongs to the officer's department, and has no assigned officer. This is an atomic claim: if two officers race, only one update should match. The other receives a conflict response rather than silently taking the same task.

Editing and audit trail

A citizen can edit only their own pending complaint. Status changes create StatusHistory records containing the entity, previous and new status, note, actor, and time. Workflow actions also create ActivityLog records. Notifications are saved separately and can be marked read.

A department-only routing action can be recorded as pending -> pending: the status did not change, but the history entry records that routing occurred.

7. Data model and MongoDB

MongoDB stores documents. Mongoose schemas add structure, types, validation, references, defaults, timestamps, and indexes. The main models are:

- User (backend/src/models/User.js): full name, unique email, passwordHash, role, phone, optional department, active state, and reset-token fields.
- Complaint (backend/src/models/Complaint.js): reference ID, title, description, category, location, status, citizen, department, assigned officer, photo/attachment URLs, resolution note, and timestamps.
- Department (backend/src/models/Department.js): unique name, description, active flag, and creation time.
- StatusHistory (backend/src/models/StatusHistory.js): complaint transition details and actor.
- ActivityLog (backend/src/models/ActivityLog.js): user action, entity, details, and time.
- Notification (backend/src/models/Notification.js): recipient, message, related entity, read state, and time.

ObjectId references connect documents, such as a complaint to its citizen, department, and officer. Indexes support common filters and sorting, including complaint citizen, department, status, creation time, user role, and notification recipient/time. Indexes improve relevant reads but add storage and write work.

Reports use MongoDB aggregation to group complaint counts by category or department on the database side. The summary endpoint uses parallel count queries.

8. API areas

Area	Main purpose
GET /api/health	Basic process health response
/api/auth	Registration, login, current user, password reset
/api/complaints	List, create, edit pending, assign, update status
/api/users	Profile operations; admin user listing and activation
/api/departments	List departments; admins may add departments
/api/uploads	Authenticated image upload
/api/reports	Admin summary and grouped reports
/api/notifications	List notifications and mark read
/api/status-histories	Status history filtered by access scope
/api/activity-logs	Admin access to recent activity records

Route definitions are under backend/src/routes/. Controller implementations are under backend/src/controllers/. Notification, activity-log, and status-history routes are grouped in activity.routes.js.

9. Uploads, notifications, email, SMS, and AI

Image uploads

backend/src/controllers/uploadController.js configures Multer disk storage, unique filenames, a 2 MB limit, and an image MIME-type check. The API stores a path/URL on the complaint rather than embedding image bytes in MongoDB. The served path is /uploads.

Notifications and external delivery

In-app notifications are MongoDB records. backend/src/utils/workflow.js creates notifications, status history, and activity logs. For workflow notifications, it starts email and SMS delivery asynchronously so provider delivery does not need to block the main request. Errors in the channel delivery path are logged.

backend/src/services/email.js supports Resend and SMTP with provider selection and fallback behavior. backend/src/services/sms.js normalizes phone numbers, identifies disabled/sandbox/live configuration, and retries some transient network failures. backend/src/services/notifyChannels.js coordinates channel delivery.

AI sidecar

The backend proxy in backend/src/middleware/aiProxy.js forwards /api/ai requests to AI_SERVICE_URL, applies an in-memory per-IP limit, times out upstream calls, and returns a service-unavailable response when it cannot connect. AI service endpoints are documented in ai-service/README.md. The core complaint workflow remains available without AI.

10. Environment variables and deployment

Configuration is environment-driven; secrets should never be hard-coded or shown during a presentation. The template is backend/.env.example.

- PORT: API listening port.
- MONGODB_URI or MONGODB_USER / MONGODB_PASSWORD / MONGODB_CLUSTER / MONGODB_DB: database connection.

- JWT_SECRET and JWT_EXPIRES_IN: signing key and token lifetime.
- CLIENT_ORIGIN: public frontend origin used by CORS and reset links.
- UPLOAD_DIR: local upload directory.
- EMAIL_PROVIDER, RESEND_API_KEY, SMTP_* and EMAIL_FROM: email channel.
- SMS_ENABLED, AT_USERNAME, AT_API_KEY, AT_SENDER_ID: Africa's Talking SMS.
- AI_SERVICE_URL: URL of the optional AI service.

The repository includes Render configuration in render.yaml, a process launcher in scripts/start-production.js, and Vercel routing in vercel.json plus api/index.js. These are distinct deployment paths. Say which one your demonstration uses, and do not claim that every path has been tested. Render starts the backend and AI sidecar together via the launcher; the Vercel API export path is different from the normal Node startup path.

11. Current limitations and credible improvements

- Uploads use server-local disk. On ephemeral hosting, files may not survive restarts or redeployments; durable object storage is a production improvement.
- AI and forgot-password rate limits use in-memory Maps. They reset on restart and are not shared across instances; a shared rate-limit store is needed for multi-instance production.
- Reference IDs are generated by counting existing records and adding one. Concurrent submissions can potentially choose the same number; a database-backed atomic sequence or collision-safe retry would be safer.
- /api/health confirms the API route responds, but does not prove MongoDB or the AI service is healthy.
- The generic error handler returns err.message as its fallback. In production, responses should avoid exposing internal error details while logs retain useful diagnostics.
- The backend package has a notification-channel smoke-test script, but that is not the same as comprehensive automated unit and integration coverage. Workflow, permissions, concurrency, and error paths deserve automated tests.

Do not present limitations as if they are already fixed. A clear answer is: "This is the current implementation; for production scaling, I would prioritize durable storage, shared rate limits, collision-safe IDs, deeper health checks, and workflow tests."

12. Presentation plan (7-10 minutes)

1. Problem and goal (45 seconds): explain why residents need a transparent way to submit and track complaints.

2. Architecture (1 minute): frontend -> Express API -> MongoDB, then optional AI and notification providers.

3. Backend structure (1 minute): entry point, app setup, routes, middleware, controllers, models, services, utilities.

4. One complete workflow (2 minutes): citizen submits; admin routes; officer claims/works; citizen receives an update.

5. Security and data integrity (1 minute): password hashing, JWT, role authorization, validation, ownership scoping, safe serialization.

6. Auditability (1 minute): status history, activity logs, notifications, and reports.

7. Configuration and deployment (1 minute): environment variable purposes and the deployment path used.

8. Limitations and next steps (1 minute): acknowledge practical improvements.

Suggested opening

> *"My part focuses on the backend of the Adama City Citizen Complaint System. It provides the API, authentication, role-based access, complaint workflow, MongoDB persistence, and notifications. The code separates routes, controllers, models, middleware, and services so HTTP handling, business rules, and external integrations have distinct responsibilities."*

Suggested end-to-end explanation

> *"A citizen submits a complaint through the API. The backend verifies the JWT, checks the user role, validates the complaint data and Adama location, then saves it as pending with a reference ID. An admin routes it to a department or an officer. An officer can work only on a task assigned to them or available in their department. Status updates are recorded, logged, and used to notify the citizen."*

13. Demo checklist

- Confirm MongoDB is available and the backend starts successfully.
- Prepare citizen, admin, and officer accounts. Know the officer's department.
- Choose one small workflow: submit a complaint, assign it, update status, then show history/notification.
- Test optional AI/email/SMS separately; the main workflow should not depend on them.
- Keep a fallback explanation ready if internet, provider credentials, or the live demo fails.
- Never expose .env values, passwords, reset tokens, or API keys on screen.
- Do not run backend/src/scripts/seed.js against data you need to keep: it deletes and recreates seeded collections.

14. Likely questions and answers

Why JWT?

It lets the client send a signed identity token with each request without a traditional server-side session. The backend verifies it and checks the account state.

Why bcrypt?

Passwords should not be stored in plain text. bcrypt stores a one-way hash and makes guessing more expensive through its work factor.

Authentication versus authorization?

Authentication establishes who the user is. Authorization decides what that user is permitted to do.

How do you prevent officers seeing other departments' cases?

Complaint queries are scoped by assignment or department, and the status update checks access to the specific record.

Why keep status history separately?

The complaint stores current state; a separate history provides an audit trail of prior and new states, actor, time, and note.

What if email or SMS fails?

The in-app notification is stored separately. External delivery is handled by channel services and should not be confused with persistence of the complaint itself.

Does AI make decisions?

No. It provides suggestions; people take action in the main complaint workflow.

What would you improve first?

Durable object storage for uploads, shared rate limiting, atomic reference IDs, deeper health checks, and automated tests for access rules and status transitions.

15. Quick glossary

Term	Plain-language meaning
API	A defined interface through which software sends requests and receives responses.
REST	A common HTTP API style using resources, URLs, methods, and status codes.
Middleware	A function in the request/response pipeline.
JWT	A signed token carrying claims that can be verified by the server.
Hash	One-way transformation used here to store passwords and reset tokens safely.
Schema	Rules describing the shape and constraints of stored documents.
ObjectId	MongoDB identifier type used for documents and references.
Aggregation	Database-side pipeline for grouping or summarizing records.
Atomic update	A database update that applies as one indivisible operation.
CORS	Browser policy controlling which web origins may read responses.
Environment variable	Configuration supplied outside source code, often per environment.

Final rehearsal reminder

Practice explaining one request from beginning to end without reading code aloud. Be able to point to its route, middleware, controller, model, and shared workflow rule. If you can explain that path clearly and honestly distinguish core behavior from optional integrations and future work, you are ready to present the backend.